RESTful API & Database Integration

Objectives

- Review Materi
- Konsep Database
- Desain Table dan ERD
- Setup Project
- Implementasi

Review

```
const StudentController = require("../controllers/StudentController");
//biasanya routing api ini disebut dengan endpoint

const express = require("express");
const router = express.Router();
router.get("/", (req, res) =>{
    res.send("Hello Express");
});
router.get("/students", StudentController.index);
router.post("/students", StudentController.store);
router.put("/students/:id", StudentController.update);
router.delete("/students/:id", StudentController.destroy);

module.exports = router;
```

```
class StudentController{
    index(req, res){
        const data = {
            message: "Menampilkan semua data",
            data: [],
        };
        res.json(data);
    }
     store(req, res){
        res.send("Menambahkan data Students");
    }
     update(req, res){
        const {id} = req.params;
        res.send(`Mengubah data Students ${id}`);
    }
     destroy(req, res){
                const {id} = req.params;
        res.send(`Menghapus data Students ${id}`);
    }
}
const object = new StudentController();

module.exports = object;
```

RESTful

- **RESTful API** adalah cara membuat sistem backend agar bisa **berkomunikasi dengan client (frontend / aplikasi lain)** menggunakan HTTP.
- Client kirim request (misal dari Postman / frontend)
- Request masuk ke server (Express.js)
- Diproses di controller
- Ambil / simpan data
- Kirim response (JSON)

Integrasi Database

- **Integrasi database** adalah proses **menghubungkan backend (Express.js) dengan database (MySQL)** agar data:
- Bisa disimpan, diambil, dikelola

Konsep Database

Masalah Data Statis

- **Data Tidak Persisten**

Setiap kali server di-restart (atau crash), semua perubahan data yang disimpan dalam variabel memori akan hilang kembali ke kondisi awal.

- **Keterbatasan Query**

Sulit melakukan pencarian, penyaringan (filtering), atau pengurutan (sorting) data yang kompleks hanya dengan array JavaScript biasa.

- **Isu Skalabilitas & Kolaborasi**

Tidak mendukung akses data secara bersamaan (concurrency) dan tidak aman untuk aplikasi dengan banyak pengguna.

Solusi: Database

Gunakan **Database Management System (DBMS)** seperti MySQL untuk menyimpan data secara permanen, terstruktur, dan aman.

Mengapa Perlu Database

- **Persistensi data**

Data tersimpan secara permanen di media penyimpanan fisik. Data tetap aman meskipun server dimatikan atau mengalami crash.

- **Konsistensi & Struktur**

Skema tabel yang jelas menjamin keseragaman tipe data (misal: integer, varchar, date) dan mencegah input yang tidak valid.

- **Kemampuan Query**

Memungkinkan pencarian, penyaringan, pengurutan, dan pengelompokan data dalam jumlah besar dengan sangat efisien.

- **Integritas Data**

Menerapkan aturan di level database seperti *Primary Key* (unik) dan relasi antar tabel untuk menjaga validitas data.

Konsep RESTful API + Database

1. Client Request

Client (Postman/Frontend) mengirim HTTP Request (GET, POST, PUT, DELETE) ke server.

2. Routing (Express)

Express menerima request dan meneruskannya ke fungsi Controller yang sesuai berdasarkan URL endpoint.

3. Controller

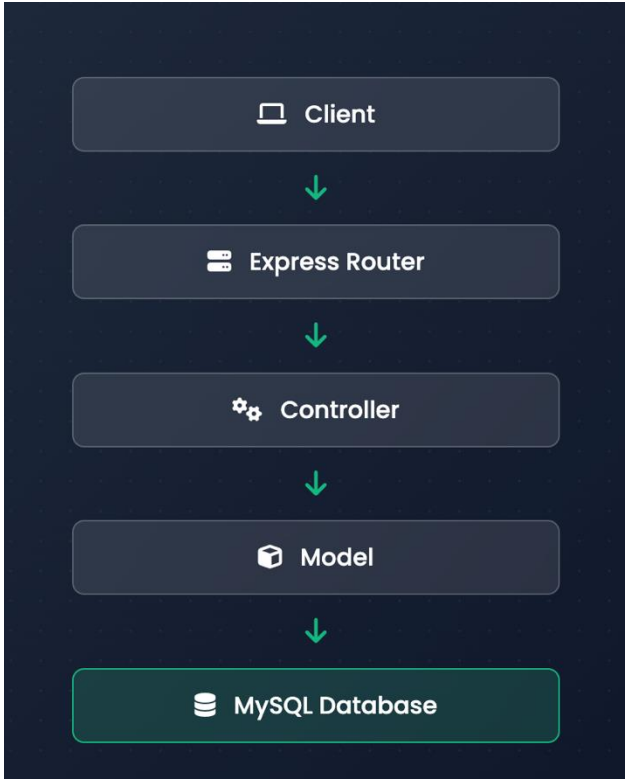
Mengatur logika aplikasi. Controller memanggil Model untuk operasi data, bukan query langsung.

4. Model

Layer yang berinteraksi langsung dengan Database. Berisi perintah SQL untuk manipulasi data.

5. Database

Menyimpan data fisik. Mengembalikan hasil query ke Model → Controller → Client sebagai JSON.



Cara Akses Database

- **Raw Query (SQL Langsung)**

Query ditulis langsung seperti di MySQL

- **Query Builder**

Query dibuat dengan fungsi JavaScript (tidak tulis SQL manual)

- **ORM (Object Relational Mapping)**

Database diperlakukan seperti object

Desain Table Database

Pengenalan MySQL

- **Struktur Tabel (Rows & Columns)**

Data disimpan dalam tabel dua dimensi. Baris (row) merepresentasikan satu entitas data, sedangkan kolom (column) adalah atribut dari data tersebut.

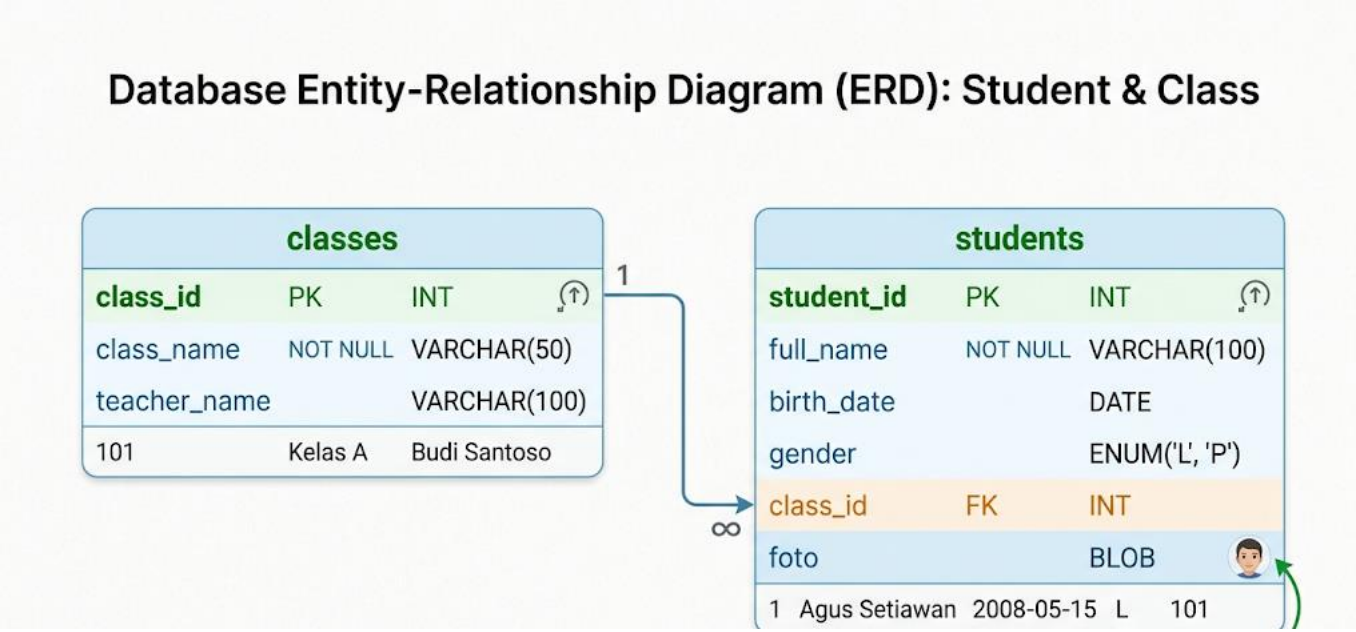
- **Tipe Data & Kunci Utama**

Setiap kolom memiliki tipe data spesifik (INT, VARCHAR, DATE). **Primary Key** digunakan sebagai identitas unik untuk setiap baris.

- **Relasi Antar Tabel**

Kekuatan utama SQL adalah kemampuan menghubungkan data antar tabel (One-to-Many, Many-to-Many) melalui Foreign Key.

Desain ERD


Database Entity-Relationship Diagram (ERD): Student & Class
classes
class_id PK INT
class_name NOT NULL VARCHAR(50)
teacher_name VARCHAR(100)
101 Kelas A Budi Santoso
1
∞
students
student_id PK INT
full_name NOT NULL VARCHAR(100)
birth_date DATE
gender ENUM('L', 'P')
class_id FK INT
foto BLOB
1 Agus Setiawan 2008-05-15 L 101

Alur Request & Response

```
1. Request
(GET /students)
          | -------------------------> |
          |                             |
          |                             |
          |                             
```

Struktur untuk database

```
express-backend/
│
├── server.js
├── config/
│   └── database.js
├── controllers/
│   └── userController.js
├── models/
│   └── userModel.js
├── routes/
│   └── userRoutes.js
├── middleware/
│   └── authMiddleware.js
├── services/
│   └── userService.js
├── utils/
│   └── helper.js
├── validators/
│   └── userValidator.js
└── public/
    └── uploads/
```

Setup Project

Setup Project

- Pastikan project sudah terinstall
- Pastikan struktur folder sudah dibuat
- Pastikan tools untuk ujicoba endpoint sudah terinstall, Misal: Postman
- Siapkan koneksi dengan MySql
- `npm install mysql2`

Create DB

- CREATE DATABASE db_students;

```sql
-- 1. Membuat Tabel Master Kelas
CREATE TABLE classes (
    class_id INT PRIMARY KEY AUTO_INCREMENT,
    class_name VARCHAR(50) NOT NULL,
    teacher_name VARCHAR(100)
);

-- 2. Membuat Tabel Siswa dengan kolom Foto
CREATE TABLE students (
    student_id INT PRIMARY KEY AUTO_INCREMENT,
    full_name VARCHAR(100) NOT NULL,
    birth_date DATE,
    gender ENUM('L', 'P'),
    class_id INT,
    photo VARCHAR(255) DEFAULT 'default_profile.jpg',
    CONSTRAINT fk_student_class
        FOREIGN KEY (class_id)
        REFERENCES classes(class_id)
        ON DELETE SET NULL
        ON UPDATE CASCADE
);
```

Config/database.js

- Buat Folder `Config/databases.js`

```javascript
const mysql = require("mysql2");

// buat koneksi
const db = mysql.createConnection({
  host: "localhost",
  user: "root",
  password: "",
  database: "db_student"
});

// test koneksi
db.connect((err) => {
  if (err) {
    console.log(" Koneksi gagal:", err);
  } else {
    console.log(" Koneksi database berhasil");
  }
});

module.exports = db;
```

Install Postman

- **Platform API (Application Programming Interface)** yang digunakan oleh pengembang untuk membangun, menguji, dan memodifikasi API secara cepat dan efisien.
- **Mengirim Request HTTP**: Kamu bisa mencoba berbagai metode seperti **GET** (mengambil data), **POST** (mengirim data), **PUT** (update), dan **DELETE** tanpa harus membuat kode *front-end* terlebih dahulu.
- Download postman https://www.postman.com/downloads/
- `Pastikan Postman sudah terinstall pada laptop kita`

Testing Koneksi

- Tambahkan routing ke app.js

```javascript
const express = require("express");
const router = require ("./routes/api");
const db = require("./config/databases");

const app = express();

app.use(express.json());
app.use(express.urlencoded());
app.use(router);

// TAMBAHAN: endpoint test database
app.get("/test-db", (req, res) => {
    db.query("SELECT 1", (err, result) => {
        if (err) {
            res.json({ message: "Koneksi database gagal" });
        } else {
            res.json({
                message: "Koneksi database berhasil",
                result: result
            });
        }
    });
});

//pindahkan routing ke routes/api.js
app.listen(3000, ()=>{
    console.log("Server berjalan di port 3000");
});
```

Implementasi

SQL Dasar isi data

```
INSERT INTO classes (class_name, teacher_name)
VALUES
('X RPL 1', 'Pak Budi'),
('X RPL 2', 'Bu Siti'),
('X TKJ 1', 'Pak Andi');
```

```
INSERT INTO students (full_name, birth_date, gender, class_id, photo) VALUES
('Ahmad Fauzi', '2005-01-15', 'L', 1, 'ahmad.jpg'),
('Siti Nurhaliza', '2005-03-20', 'P', 2, 'siti.jpg'),
('Budi Santoso', '2004-12-10', 'L', 1, 'budi.jpg');
```

Models

Buat file models untuk memanggil table students

```
const db = require("../config/databases");

class Student {

  static getAll(callback){
    const sql = "SELECT * FROM students";
    db.query(sql, callback);
  }

  static getById(id, callback){
    const sql = "SELECT * FROM students WHERE student_id = ?";
    db.query(sql, [id], callback);
  }


}

module.exports = Student;
```

Models Create, Update, Delete

```
14
15    static create(data, callback){
16      const sql = `
17        INSERT INTO students (full_name, birth_date, gender, class_id, photo)
18        VALUES (?, ?, ?, ?, ?)
19      `;
20      db.query(sql, [
21        data.full_name,
22        data.birth_date,
23        data.gender,
24        data.class_id,
25        data.photo
26      ], callback);
27    }
28
29    static update(id, data, callback){
30      const sql = `
31        UPDATE students
32        SET full_name=?, birth_date=?, gender=?, class_id=?, photo=?
33        WHERE student_id=?
34      `;
35      db.query(sql, [
36        data.full_name,
37        data.birth_date,
38        data.gender,
39        data.class_id,
40        data.photo,
41        id
42      ], callback);
43    }
44
45    static delete(id, callback){
46      const sql = "DELETE FROM students WHERE student_id=?";
47      db.query(sql, [id], callback);
48    }
49
50  }
```

Controllers

Buat file controllers untuk mengelola table students

```
const Student = require("../models/Students");

class StudentController {

  // GET ALL
  index(req, res){
    Student.getAll((err, results)=>{
      if(err){
        return res.json({message: "Gagal ambil data"});
      }

      res.json({
        message: "Berhasil ambil semua data",
        data: results
      });
    });
  }

}

module.exports = new StudentController();
```

Controllers Store, Show Update, Delete

```
19  // GET BY ID
20  show(req, res){
21    const {id} = req.params;
22
23    Student.getById(id, (err, results)=>{
24      if(err){
25        return res.json({message: "Data tidak ditemukan"});
26      }
27
28      res.json({
29        message: "Detail student",
30        data: results[0]
31      });
32    });
33  }
34
35  // CREATE
36  store(req, res){
37    const data = req.body;
38
39    Student.create(data, (err)=>{
40      if(err){
41        return res.json({message: "Gagal tambah data"});
42      }
43
44      res.json({
45        message: "Data berhasil ditambahkan",
46        data: data
47      });
48    });
49  }
```

```
50
51  //  UPDATE
52  update(req, res){
53    const {id} = req.params;
54    const data = req.body;
55
56    Student.update(id, data, (err)=>{
57      if(err){
58        return res.json({message: "Gagal update data"});
59      }
60
61      res.json({
62        message: "Data berhasil diupdate"
63      });
64    });
65  }
66
67  // DELETE
68  destroy(req, res){
69    const {id} = req.params;
70
71    Student.delete(id, (err)=>{
72      if(err){
73        return res.json({message: "Gagal hapus data"});
74      }
75
76      res.json({
77        message: "Data berhasil dihapus"
78      });
79    });
80  }
81
```

Routes/Api

Sesuaikan kode yang ada di routes/api

```
const StudentController = require("../controllers/StudentController");
//biasanya routing api ini disebut dengan endpoint

const express = require("express");
const router = express.Router();
router.get("/", (req, res) =>{
    res.send("Hello Express");
});
router.get("/students", StudentController.index);
router.get("/students/:id", StudentController.show);
router.post("/students", StudentController.store);
router.put("/students/:id", StudentController.update);
router.delete("/students/:id", StudentController.destroy);

module.exports = router;
```

Installation library

Instalasi Dotenv melalui NPM.

- npm install dotenv
- menyimpan konfigurasi penting di luar kode program
- Menyimpan Data Sensitif

Perubahan pada app.js dan databases

```javascript
require("dotenv").config(); //  TAMBAHAN

const express = require("express");
const router = require ("./routes/api");

const app = express();

app.use(express.json());
app.use(express.urlencoded());
app.use(router);

app.listen(process.env.PORT, ()=>{
    console.log(`Server berjalan di port ${process.env.PORT}`);
});
```

```javascript
const mysql = require("mysql2");

const db = mysql.createConnection({
  host: process.env.DB_HOST,
  user: process.env.DB_USER,
  password: process.env.DB_PASS,
  database: process.env.DB_NAME
});

db.connect((err) => {
  if (err) {
    console.log("❌ Koneksi gagal:", err);
  } else {
    console.log("✅ Koneksi database berhasil");
  }
});

module.exports = db;
```

Overview

Task

- Masuk Sprint minggu ke-3
- Setiap kelompok memastikan **database sudah dibuat sesuai dengan ERD yang telah dirancang pada sprint sebelumnya**
- Setiap kelompok memastikan **koneksi backend ke database sudah berhasil dilakukan**
- Melakukan konfigurasi koneksi database pada project backend menggunakan **Express.js**
- Backend harus sudah dapat **terhubung ke database tanpa error**
- Melakukan uji koneksi database (test query sederhana seperti SELECT)
- Setiap kelompok mulai membuat **model / query database** untuk masing-masing table
- Menghubungkan backend dengan database menggunakan ORM / query builder (optional)
- Setiap anggota kelompok mulai mengerjakan fitur **CRUD sesuai pembagian tugas**
- Masukan progres pengerjaan kedalam pembagian task/backlog sesuai template sprint yang sudah disediakan
- Kumpulkan laporan sesuai format yang sudah disediakan di Elena
- Kumpulkan laporan kepada asisten dosen dan Elena
- Buat repository kelompok dan collaborate akun github masing masing **asdos dan akun arip86**

Referensi

- Express.js: Documentation